CORRIGÉ · PROBLÈME · ANNALES SPÉ

Relations d'ordre, chaînes et antichaînes

Énoncé

1. Relation d'ordre et tri topologique

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

1. Écrire `check_reflexivity (r:order) : bool` dans le paradigme **impératif**.
2. Écrire `check_reflexivity_bis (r:order) : bool` dans le paradigme **fonctionnel**.
3. Écrire `check_antisymmetry (r:order) : bool`.
4. Écrire `check_transitivity (r:order) : bool`.
5. Écrire `is_order (r:order) : bool` à partir de 1, 3 et 4.

Le graphe de l'ordre a pour arcs les (v_i, v_j) tels que $v_i \trianglelefteq v_j$; le degré sortant d'un sommet est le nombre d'arcs qui en partent.

1. Écrire `outdegrees (r:order) : int array` (degrés sortants).
2. Écrire `argmin (d:int array) : int` (un indice d'une valeur minimale).

Un tri topologique est une permutation σ de $[\![0, n-1]\!]$ telle que $v_i \trianglelefteq v_j$ entraîne $\sigma(i) \leqslant \sigma(j)$; v_i est alors le sommet de numéro $\sigma(i)$.

1. Montrer qu'il existe une constante δ_0 telle que, pour tout tri topologique σ, le sommet de numéro $n-1$ est de degré sortant δ_0 et les autres de degré sortant $\geqslant \delta_0$.
2. Écrire `topological_sort (r:order) : int array` qui attribue les numéros par ordre décroissant en s'appuyant sur la question 8.
3. Énoncer des invariants qui prouvent la correction de `topological_sort`.
4. Complexité en temps de `topological_sort` en fonction de n ; comment la qualifier par rapport à la taille de l'entrée ?

2. Chaînes, antichaînes et tas

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type`

`order = bool array array`).

Une chaîne est une partie C telle que deux éléments quelconques c, c' vérifient $c \trianglelefteq c'$ ou $c' \trianglelefteq c$ (représentée par une `int list`). Code donné :

```
let rec is_chain (r:order) (c:int list) : bool =
  match c with
  | [] -> true
  | v::cc -> List.for_all (fun x -> x > v || x < v) cc
             && is_chain r cc
```

1. Confirmer ou réfuter : `is_chain r c` teste si `c` est une chaîne pour `r`. Dans le second cas, corriger la fonction.
2. Complexité de `is_chain` en fonction de la longueur γ de `c`.

On veut `is_chain_bis` en $O(\gamma \log \gamma)$ avec une structure mutable de capacité bornée : `init : order -> int -> hp`, `is_empty : hp -> bool`, `push : hp -> int -> bool`, `pop : hp -> bool` ; `push` et `pop` peuvent échouer, ce que signale leur valeur de retour. La fonction insère tous les éléments de `c` puis les retire tous, et renvoie la conjonction des succès.

1. Compléter la description du type `hp` en nommant une structure de données (avec son invariant) pour que `is_chain_bis` respecte la spécification en $O(\gamma \log \gamma)$. Justifier.

Une antichaîne est une partie dont les éléments sont deux à deux incomparables.

1. Modifier `is_chain` en `is_antichain` ; en déduire sa complexité.
2. Confirmer ou réfuter l'existence d'une fonction `is_antichain_bis` en $O(\alpha \log^2 \alpha)$, où α est la longueur de la liste.

3. Couverture minimale par des chaînes

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

Une couverture est une famille $(C_\ell)_{1 \leqslant \ell \leqslant \lambda}$ de chaînes dont la réunion est V.

1. Si (C_ℓ) est une couverture et A une antichaîne de cardinal α, montrer que $\alpha \leqslant \lambda$.

Graphe des poursuites : graphe biparti P sur $W_0 = \{w_i\}$ et $W_1 = \{w_{n+i}\}$ (deux copies de V), avec une arête (w_i, w_{n+j}) dès que $i \neq j$ et $v_i \trianglelefteq v_j$. Représentation : `type bipartite = int list array` de longueur $2n$, chaque arête notée deux fois. Un couplage est un ensemble d'arêtes d'extrémités distinctes.

1. Écrire `bipartite_of_order (r:order) : bipartite`.
2. Pour un couplage M, on prend la partition (C_ℓ) de V la plus fine possible telle que $(w_i, w_{n+j}) \in M$ impose que v_i et v_j soient dans la même part. Montrer que chaque C_ℓ est une chaîne.

Un couplage est codé par `m : int array` de longueur n : `m.(j) = i` si $(w_i, w_{n+j}) \in M$, et -1 si w_{n+j} n'est pas couvert.

1. Écrire `chains_of_matching (m:int array) : int list list`.
2. Une couverture par des chaînes disjointes étant fixée, construire un couplage qui la redonne par la question 19.
3. Montrer que la construction de 19 donne une couverture par un nombre minimum de chaînes si et seulement si M est de cardinal maximum.
4. Écrire `maximum_matching (p:bipartite) : int array`.
5. Écrire `minimum_cover (r:order) : int list list`.
6. Complexité de `minimum_cover` en fonction de n.

Décoder l'énoncé

Ce que l'énoncé te donne.

- La matrice est de taille $n \times n$: lire toute l'entrée coûte déjà n^2.
- Réflexivité : chaque sommet a une boucle, donc un degré sortant $\geqslant 1$.
- `x > v || x < v` compare les **numéros** des sommets, pas les éléments pour la relation `r`.
- La transitivité permet de déduire des comparaisons sans les tester toutes.
- Dans un couplage, chaque w_i a au plus un partenaire « à droite » et chaque w_{n+j} au plus un « à gauche » : on construit des chemins.

Ce qu'on te demande. Vérifier les axiomes d'un ordre, puis numéroter les sommets en retirant à chaque fois un élément maximal.

Corrigé

Question 1

```
let check_reflexivity (r:order) : bool =
  let n = Array.length r in
  let ok = ref true in
  for i = 0 to n - 1 do
    if not r.(i).(i) then ok := false   (* v_i n'est pas en relation avec lui-même *)
  done;
  !ok
```

Style impératif : variable mutable (référence) et boucle. Complexité $O(n)$.

Question 2

```
let check_reflexivity_bis (r:order) : bool =
  let n = Array.length r in
  let rec from i = i >= n || (r.(i).(i) && from (i + 1)) in
  from 0
```

Aucune variable mutable : la fonction récursive `from i` vaut « tous les $r_{k,k}$ pour $k \geqslant i$ sont vrais ». Le `&&` paresseux arrête dès le premier échec. Variante tout aussi fonctionnelle : `Array.for_all (fun b -> b) (Array.mapi (fun i ligne -> ligne.(i)) r)`.

Question 3

```
let check_antisymmetry (r:order) : bool =
  let n = Array.length r in
  let ok = ref true in
  for i = 0 to n - 1 do
    for j = 0 to n - 1 do
      if i <> j && r.(i).(j) && r.(j).(i) then ok := false
    done
  done;
  !ok
```

Comme les v_i sont distincts, $v_i = v_j$ équivaut à $i = j$: l'antisymétrie interdit d'avoir les deux relations pour $i \neq j$. Complexité $O(n^2)$.

Question 4

```
let check_transitivity (r:order) : bool =
  let n = Array.length r in
  let ok = ref true in
  for i = 0 to n - 1 do
    for j = 0 to n - 1 do
      if r.(i).(j) then
        for k = 0 to n - 1 do
          if r.(j).(k) && not r.(i).(k) then ok := false   (* contre-exemple *)
        done
    done
  done;
  !ok
```

On teste tous les triplets (i, j, k) : complexité $O(n^3)$.

Question 5

```
let is_order (r:order) : bool =
  check_reflexivity r && check_antisymmetry r && check_transitivity r
```

Question 6

```
let outdegrees (r:order) : int array =
  let n = Array.length r in
  let d = Array.make n 0 in
  for i = 0 to n - 1 do
    for j = 0 to n - 1 do
      if r.(i).(j) then d.(i) <- d.(i) + 1   (* arc (v_i, v_j) *)
    done
  done;
  d
```

Les arcs issus de v_i sont les (v_i, v_j) avec $v_i \trianglelefteq v_j$, boucle (v_i, v_i) comprise. Complexité $O(n^2)$.

Question 7

```
let argmin (d:int array) : int =
  let best = ref 0 in
  for j = 1 to Array.length d - 1 do
    if d.(j) < d.(!best) then best := j
  done;
  !best
```

On suppose `d` non vide. Invariant : après le tour j, `d.(!best)` est le minimum de `d.(0..j)`.

Question 8

On montre que $\delta_0 = 1$ convient.

Tous les degrés sont $\geqslant 1$: par réflexivité, chaque v_i a la boucle (v_i, v_i).

Le sommet de numéro $n-1$ **a un degré exactement** 1 : soit $u = v_{\sigma^{-1}(n-1)}$ et soit un arc (u, v_j), c'est-à-dire $u \trianglelefteq v_j$. Par définition du tri topologique, $\sigma(j) \geqslant \sigma(\sigma^{-1}(n-1)) = n-1$, donc $\sigma(j) = n-1$ (c'est le plus grand numéro), donc $v_j = u$ (σ est une bijection). Le seul arc issu de u est sa boucle.

> $\delta_0 = 1$: le sommet de numéro $n-1$ est de degré sortant 1 (c'est un élément maximal), les autres de degré $\geqslant 1$.

Conséquence utile : un sommet de degré sortant minimal (égal à 1) est un élément maximal ; c'est lui qu'on numérote en dernier.

Question 9

Idée : d'après Q8 appliquée à l'ensemble des sommets restants, un sommet de degré sortant minimal (dans le graphe restant) est maximal. On lui donne le plus grand numéro disponible, on le retire, et on recommence. Retirer v diminue de 1 le degré de chaque sommet restant $u \neq v$ tel que $u \trianglelefteq v$; on met le degré de v à `max_int` pour ne plus le choisir.

```
let topological_sort (r:order) : int array =
  let n = Array.length r in
  let d = outdegrees r in              (* degrés sortants dans le graphe restant *)
  let sigma = Array.make n 0 in
  for k = n - 1 downto 0 do
    let v = argmin d in                (* sommet maximal parmi les restants (Q8) *)
    sigma.(v) <- k;                    (* il reçoit le plus grand numéro libre *)
    d.(v) <- max_int;                  (* on le retire *)
    for u = 0 to n - 1 do
      if d.(u) <> max_int && r.(u).(v) then d.(u) <- d.(u) - 1   (* arc (u,v) supprimé *)
    done
  done;
  sigma
```

Question 10

Notons R_k l'ensemble des sommets numérotés avant le tour k (au départ, $R_{n-1} = \emptyset$). Au début du tour k :

- **(I1)** σ réalise une bijection de R_k sur $[\![k+1, n-1]\!]$;
- **(I2)** pour tout $u \notin R_k$: `d.(u)` $= \text{card}\{w \notin R_k : v_u \trianglelefteq v_w\}$ (degré sortant dans le graphe restant), et `d.(u)` $=$ `max_int` pour $u \in R_k$;
- **(I3)** si $v_i \trianglelefteq v_j$ et $i \in R_k$, alors $j \in R_k$ et $\sigma(i) \leqslant \sigma(j)$ (aucun sommet numéroté n'est en dessous d'un sommet restant).

Initialisation : $R = \emptyset$, `d` $=$ degrés de Q6 : vrai.

Conservation : le sommet restant est un ensemble ordonné (restriction de l'ordre). D'après Q8 appliquée à lui, le minimum des degrés restants vaut 1 et le sommet v choisi est maximal parmi les restants : si $v \trianglelefteq w$ avec w restant, alors $w = v$. On lui donne le numéro k :

- (I1) : σ est une bijection de $R_k \cup \{v\}$ sur $[\![k, n-1]\!]$;
- (I3) : si $v \trianglelefteq w$, alors soit $w = v$, soit $w \in R_k$, de numéro $> k$; pour les anciens sommets numérotés, rien ne change ;
- (I2) : retirer v enlève exactement un arc (u, v) à chaque u restant tel que $v_u \trianglelefteq v$; c'est la mise à jour faite.

Conclusion : après le tour $k = 0$, $R = V$, σ est une permutation de $[\![0, n-1]\!]$ (I1) et respecte l'ordre (I3) : c'est un tri topologique.

Question 11

`outdegrees` coûte $O(n^2)$. Puis n tours, chacun avec un `argmin` en $O(n)$ et une mise à jour en $O(n)$: $O(n^2)$.

Complexité : $\Theta(n^2)$.

L'entrée `r` occupe n^2 cases : la complexité est **linéaire en la taille de l'entrée**. On ne peut pas faire mieux, puisqu'il faut au moins lire la matrice.

Question 12

Réfutation : la fonction compare les entiers `x` et `v` avec l'ordre usuel $\leqslant$, et non avec la relation `r` (qui n'est même pas utilisée). Pour deux sommets distincts, `x > v || x < v` est toujours vrai : `is_chain r c` renvoie `true` pour toute liste sans doublon.

Contre-exemple : l'ordre « égalité » sur $V = \{v_0, v_1\}$ (`r` = matrice identité) ; $\{v_0, v_1\}$ n'est pas une chaîne, mais `is_chain r [0;1]` renvoie `true`.

Correction : tester la comparabilité pour `r`.

```
let rec is_chain (r:order) (c:int list) : bool =
  match c with
  | [] -> true
  | v :: cc -> List.for_all (fun x -> r.(v).(x) || r.(x).(v)) cc
               && is_chain r cc
```

Preuve (récurrence sur la longueur) : la liste vide est une chaîne. Une liste `v :: cc` est une chaîne si et seulement si `v` est comparable à chaque élément de `cc` (test du `for_all`) et `cc` est une chaîne (appel récursif, correct par hypothèse de récurrence) : ce sont exactement toutes les paires.

Question 13

Sur une liste de longueur ℓ, `for_all` fait au plus $\ell - 1$ tests en $O(1)$, puis l'appel récursif porte sur une liste de longueur $\ell - 1$. Dans le pire cas (une chaîne : aucun arrêt anticipé) :

$$\sum_{\ell=1}^{\gamma} (\ell - 1) = \frac{\gamma(\gamma - 1)}{2}.$$

Complexité : $O(\gamma^2)$.

Question 14

Structure : `hp` est un **tas binaire** (file de priorité) de capacité γ, rangé dans un tableau, dont les comparaisons se font avec `r`. Invariant de bonne constitution : pour tout nœud p de fils f, on a $v_p \trianglelefteq v_f$ (le parent est plus petit que ses fils pour `r`).

Opérations :

- `push q i` : on place i en dernière position et on le fait remonter. Chaque comparaison avec le parent utilise `r` ; si les deux éléments sont **incomparables**, `push` renvoie `false`.
- `pop q` : on retire la racine, on met le dernier élément à la racine et on le fait descendre en le comparant à ses fils (et les fils entre eux). Si une comparaison porte sur deux incomparables, `pop` renvoie `false`.

Correction :

- si `c` est une chaîne, toutes les comparaisons réussissent : la fonction renvoie `true` ;
- si la fonction renvoie `true`, l'invariant est vrai à chaque étape. La racine est alors plus petite que tous les éléments du tas (transitivité le long des branches), et le nouvel élément remonté à la racine après un `pop` était un descendant de l'ancienne racine. Les éléments sortent donc dans l'ordre $x_1 \trianglelefteq x_2 \trianglelefteq \cdots \trianglelefteq x_\gamma$; par transitivité, deux éléments quelconques sont comparables : `c` est une chaîne ;
- la fonction ne renvoie `false` que si elle a trouvé deux éléments de `c` incomparables, auquel cas `c` n'est pas une chaîne.

Complexité : γ insertions et γ retraits, chacun en $O(\log \gamma)$ comparaisons (hauteur du tas) : $O(\gamma \log \gamma)$. C'est un tri par tas qui vérifie au passage que tout est comparable.

Question 15

```
let rec is_antichain (r:order) (a:int list) : bool =
  match a with
  | [] -> true
  | v :: aa -> List.for_all (fun x -> not r.(v).(x) && not r.(x).(v)) aa
               && is_antichain r aa
```

La structure du code est identique à celle de `is_chain` (Q12) : seul le prédicat testé change, toujours en $O(1)$. Le décompte de Q13 s'applique tel quel : complexité $O(\alpha^2)$.

Question 16

Réfutation : tout algorithme correct doit consulter `r` pour **chacune** des $\frac{\alpha(\alpha-1)}{2}$ paires, donc faire $\Omega(\alpha^2)$ opérations, ce qui est plus que $O(\alpha \log^2 \alpha)$.

Argument d'adversaire : on part de l'ordre « égalité » ($v_i \trianglelefteq v_j \iff i = j$), pour lequel la liste `a` est une antichaîne, et l'algorithme doit répondre `true`. Supposons qu'il ne consulte jamais `r.(a).(b)` ni `r.(b).(a)` pour une certaine paire $\{a, b\}$ de la liste. On change alors `r` en ajoutant seulement $v_a \trianglelefteq v_b$. C'est encore une relation d'ordre (réflexive, antisymétrique, et transitive car il n'y a pas d'autre relation à enchaîner). L'algorithme fait exactement les mêmes lectures, donc répond encore `true`, alors que `a` n'est plus une antichaîne : contradiction.

Pourquoi c'est différent pour les chaînes : dans une chaîne, la transitivité permet de déduire des comparaisons sans les lire (c'est ce qu'exploite le tas). Pour l'incomparabilité, rien ne se déduit.

Question 17

Chaque élément $a \in A$ appartient à au moins une chaîne de la couverture ; on en choisit une, notée $C_{\ell(a)}$. L'application $a \mapsto \ell(a)$ va de A dans $[\![1, \lambda]\!]$.

Elle est **injective** : si $\ell(a) = \ell(a')$ avec $a \neq a'$, alors a et a' sont dans la même chaîne, donc comparables, ce qui contredit le fait que A est une antichaîne.

$$\alpha = \operatorname{card} A \leqslant \operatorname{card}[\![1, \lambda]\!] = \lambda.$$

Question 18

```
let bipartite_of_order (r:order) : bipartite =
  let n = Array.length r in
  let p = Array.make (2 * n) [] in
  for i = 0 to n - 1 do
    for j = 0 to n - 1 do
      if i <> j && r.(i).(j) then begin
        p.(i) <- (n + j) :: p.(i);       (* arête w_i -- w_{n+j} vue de w_i *)
        p.(n + j) <- i :: p.(n + j)      (* et vue de w_{n+j} *)
      end
    done
  done;
  p
```

Complexité $O(n^2)$.

Question 19

Graphe des liens : on relie $v_i \to v_j$ quand $(w_i, w_{n+j}) \in M$. Comme M est un couplage :

- w_i est extrémité d'au plus une arête : chaque v_i a au plus un **successeur** ;
- w_{n+j} est extrémité d'au plus une arête : chaque v_j a au plus un **prédécesseur**.

La partition la plus fine qui met dans la même part les extrémités de chaque lien (c'est celle qui maximise λ) est celle des composantes connexes de ce graphe. Chaque composante est donc un chemin ou un cycle.

Pas de cycle : un cycle $v_{i_1} \to v_{i_2} \to \ldots \to v_{i_p} \to v_{i_1}$ ($p \geqslant 2$, sommets distincts) donnerait par transitivité $v_{i_1} \trianglelefteq v_{i_2}$ et $v_{i_2} \trianglelefteq v_{i_1}$, donc $v_{i_1} = v_{i_2}$ par antisymétrie, ce qui est absurde.

Conclusion : chaque part est un chemin $v_{i_1} \to v_{i_2} \to \ldots \to v_{i_p}$ avec $v_{i_1} \trianglelefteq v_{i_2} \trianglelefteq \cdots \trianglelefteq v_{i_p}$. Par transitivité, deux éléments quelconques de la part sont comparables : **chaque C_ℓ est une chaîne**.

Question 20

`m.(j) = i` signifie que v_i a pour successeur v_j. Les sommets sans prédécesseur (`m.(j) = -1`) sont les débuts des chaînes ; on suit les successeurs.

```
let chains_of_matching (m:int array) : int list list =
  let n = Array.length m in
  let succ = Array.make n (-1) in
  Array.iteri (fun j i -> if i >= 0 then succ.(i) <- j) m;   (* lien v_i -> v_j *)
  let rec follow v = if v = -1 then [] else v :: follow succ.(v) in
  let chains = ref [] in
  for j = 0 to n - 1 do
    if m.(j) = -1 then chains := follow j :: !chains     (* v_j débute une chaîne *)
  done;
  !chains
```

Chaque sommet est dans exactement un chemin (Q19), et chaque chemin a exactement un début : on obtient toutes les chaînes, une seule fois chacune. Complexité $O(n)$.

Question 21

Soit une couverture par des chaînes disjointes. On range chaque chaîne C_ℓ dans l'ordre croissant : $c_1 \trianglelefteq c_2 \trianglelefteq \cdots \trianglelefteq c_p$ (possible car tous ses éléments sont comparables). On met dans M les arêtes $(w_{c_t}, w_{n+c_{t+1}})$ pour $1 \leqslant t < p$, pour toutes les chaînes.

- Ce sont des arêtes du graphe des poursuites : $c_t \neq c_{t+1}$ et $v_{c_t} \trianglelefteq v_{c_{t+1}}$.
- C'est un couplage : dans une chaîne, chaque élément a au plus un suivant et au plus un précédent, et les chaînes sont disjointes.
- La construction de Q19 relie exactement les éléments consécutifs de chaque chaîne : ses parts sont les C_ℓ.

On a de plus $|M| = \sum_\ell (|C_\ell| - 1) = n - \lambda$.

Question 22

Nombre de chaînes : dans Q19, les parts sont des chemins ; un chemin à p sommets contient $p - 1$ liens. En sommant sur les parts : $n - \lambda = |M|$, soit

$$\lambda = n - |M|.$$

Couvertures disjointes : on peut toujours rendre une couverture disjointe sans augmenter son nombre de chaînes (on retire un élément des chaînes où il apparaît en double ; une partie d'une chaîne est une chaîne). Le minimum $\lambda_{\min}$ est donc atteint par une couverture disjointe, et par Q21 celle-ci provient d'un couplage de cardinal $n - \lambda_{\min}$.

Équivalence :

- si $|M|$ est maximum, alors $n - |M| \leqslant n - |M'|$ pour tout couplage M', en particulier pour

celui de Q21 : $\lambda = n - |M| \leqslant \lambda_{\min}$, donc $\lambda = \lambda_{\min}$;

- réciproquement, si $\lambda = n - |M| = \lambda_{\min}$, un couplage plus grand M' donnerait une couverture de $n - |M'| < \lambda_{\min}$ chaînes, ce qui est impossible.

$$\lambda_{\min} = n - |M|_{\max}.$$

Question 23

Méthode (algorithme de Kuhn) : on traite les sommets w_i de W_0 un par un. Pour w_i, on cherche un voisin w_{n+j} non encore visité ; s'il est libre, on le couple à w_i ; sinon, on essaie de recaser son partenaire actuel `m.(j)` ailleurs, récursivement. Une réussite correspond à un chemin augmentant, qu'on inverse.

```
let maximum_matching (p:bipartite) : int array =
  let n = Array.length p / 2 in
  let m = Array.make n (-1) in              (* m.(j) = partenaire de w_{n+j} *)
  (* essaie d'attribuer un partenaire à w_i ; seen évite de reboucler *)
  let rec augment i seen =
    List.exists (fun w ->
      let j = w - n in
      if seen.(j) then false
      else begin
        seen.(j) <- true;
        if m.(j) = -1 || augment m.(j) seen then (m.(j) <- i; true)
        else false
      end) p.(i)
  in
  for i = 0 to n - 1 do
    ignore (augment i (Array.make n false))
  done;
  m
```

`List.exists` s'arrête au premier succès. **Correction** : chaque recherche réussie augmente $|M|$ de 1 ; quand plus aucune recherche ne réussit, il n'existe plus de chemin augmentant, et le couplage est maximum par le théorème de Berge.

Question 24

```
let minimum_cover (r:order) : int list list =
  chains_of_matching (maximum_matching (bipartite_of_order r))
```

D'après Q22, un couplage maximum donne une couverture par un nombre minimum de chaînes (disjointes).

Question 25

- `bipartite_of_order` : $O(n^2)$.
- `maximum_matching` : n recherches ; chacune visite chaque w_{n+j} au plus une fois et parcourt au plus une fois chaque liste d'adjacence, soit $O(n + \text{nombre d'arêtes}) = O(n^2)$. Total $O(n^3)$.
- `chains_of_matching` : $O(n)$.

Complexité de `minimum_cover` : $O(n^3)$.

Le piège

1. En Q9, ne pas mettre à jour les degrés après avoir retiré un sommet : le deuxième sommet choisi ne serait plus forcément maximal parmi les restants.
2. En Q12, croire la fonction correcte parce qu'elle « compare » : elle compare les numéros des sommets avec $\leqslant$, et non les éléments pour l'ordre `r`.
3. En Q22, oublier de justifier qu'une couverture minimale peut être prise avec des chaînes disjointes : sans cela, Q21 ne s'applique pas.

La méthode

1. Tri topologique « par la fin » : retirer à chaque étape un élément maximal (degré sortant 1 grâce à la réflexivité), en maintenant les degrés du graphe restant.
2. Borne inférieure de complexité : argument d'adversaire. Si l'algorithme ne lit pas une donnée, on la modifie sans qu'il s'en aperçoive.
3. Couverture minimale par des chaînes (Dilworth) : graphe biparti « i peut précéder j », couplage maximum, puis $\lambda_{\min} = n - |M|$.

Rappels de cours utilisés

- **Relation d'ordre.** Réflexive ($x \trianglelefteq x$), antisymétrique ($x \trianglelefteq y$ et $y \trianglelefteq x$ donnent $x = y$), transitive ($x \trianglelefteq y \trianglelefteq z$ donne $x \trianglelefteq z$). Une **chaîne** est une partie où deux éléments sont toujours comparables ; une **antichaîne**, une partie où deux éléments distincts ne le sont jamais.
- **Invariant de boucle.** Propriété vraie avant la boucle, conservée par chaque tour ; à la sortie, avec la condition d'arrêt, elle donne le résultat. C'est l'outil standard pour prouver la correction d'un programme itératif.
- **Tas binaire.** Arbre binaire presque complet rangé dans un tableau (fils de i en $2i + 1$ et $2i + 2$), avec l'invariant : **tout nœud est plus petit que ses fils**. Insertion (remontée) et retrait de la racine (descente) en $O(\log n)$ comparaisons.
- **Couplage et chemins augmentants.** Un couplage est un ensemble d'arêtes deux à deux sans extrémité commune. Un chemin augmentant alterne arêtes hors couplage et arêtes du couplage entre deux sommets libres. **Théorème de Berge** : un couplage est maximum si et

seulement s'il n'admet aucun chemin augmentant.